

GUIDE COMPLET POUR L'EXAMEN J2EE

Synthese des cours et TPs

4IIR - Semestre 2

15 juin 2026

Table des matières

Note importante	3
1 PREPARATION DE L'ENVIRONNEMENT (TP1)	3
2 ARCHITECTURE JAVA EE (Cours)	3
2.1 Definition de Java EE	3
3 LES SERVIETS (Cours + TPs 2, 3, 4, 5)	4
3.1 Cycle de vie d'une Servlet - TRES IMPORTANT	4
3.2 Configuration d'une Servlet	4
3.3 Les methodes HTTP - A COMPARER	5
3.4 Les objets implicites	5
3.5 forward() vs sendRedirect() - A CONNAITRE	5
4 JSP (JavaServer Pages) (Cours + TP4)	6
4.1 Difference Servlet vs JSP	6
4.2 Les Tags JSP	6
4.3 Les objets implicites JSP	6
4.4 JSTL	6
5 ARCHITECTURE MVC (Cours + TP4, TP5)	7
5.1 Definition - QUESTION FREQUENTE	7
5.2 Role de chaque couche	7
5.3 Pourquoi 3 couches ? - QUESTION COURS	7
6 JDBC (Cours)	8
6.1 Les 4 classes principales	8
6.2 Statement vs PreparedStatement - TRES IMPORTANT	8
6.3 Gestion des transactions	8
7 JPA (Cours + TP8)	9
7.1 JPA vs JDBC vs Hibernate - COMPARAISON	9
7.2 Annotations JPA fondamentales	9
7.3 Strategies @GeneratedValue - COMPARER	9
7.4 Relations JPA	9
7.5 Strategies d'heritage - CODE A SAVOIR	10

7.6	Role d'EntityManager	10
7.7	Le fichier persistence.xml	10
8	SPRING BOOT (Cours + TP6, TP7, TP8)	11
8.1	Definition	11
8.2	Starters Spring Boot - A CONNAITRE	11
8.3	L'annotation @SpringBootApplication	11
8.4	Les annotations Spring - QCM FREQUENT	12
8.5	Difference @Controller vs @RestController	12
8.6	@Autowired - COMPRENDRE LE PRINCIPE	12
8.7	Couche Repository avec Spring Data JPA - TRES IMPORTANT	13
8.8	Le DTO (Data Transfer Object) - IMPORTANT	13
9	CODE DE STATUT HTTP - QCM	14
10	SESSIONS HTTP (TP3-2)	14
10.1	Recuperer / creer une session	14
10.2	Utiliser la session	14
11	COLLABORATION ENTRE SERVLETS (TP3-2)	14
11.1	getRequestDispatcher et forward()	14
11.2	include()	15
11.3	Difference forward() vs include()	15
12	FICHIER web.xml (EXAMEN)	15
13	RESUME - CHECKLIST POUR L'EXAMEN	16
13.1	Questions de cours - PREPAREZ CES REPONSES	16
13.2	SCHEMAS A SAVOIR DESSINER	16
13.3	CODES A SAVOIR ECRIRE	17
13.4	Tableau recapitulatif des differences	17

Note importante

A retenir pour l'examen

Structure de l'examen :

- **Questions de cours** : definitions, differences, explications
- **Questions de code** :
 - Expliquer un code (fonctionnement, role des annotations)
 - Corriger un code (erreurs, annotations manquantes)
 - Completer un code (lignes manquantes, strategies d'heritage, relations JPA)

1 PREPARATION DE L'ENVIRONNEMENT (TP1)

Element	Ce qu'il faut savoir
JDK 17	Installer et configurer JAVA_HOME
Apache Tomcat	Installation, configuration, modification du port (8080 → 9090)
Fichier tomcat-users.xml	Configurer un compte admin (manager-gui)
Eclipse	Configuration de Tomcat, ajout de la JDK, vue Servers
Lombok	Installation, ajout dans eclipse.ini (javaagent)
Demarrage/arret	Via Eclipse ou startup.bat

2 ARCHITECTURE JAVA EE (Cours)

2.1 Definition de Java EE

Definition

Java EE (Jakarta EE) est une plateforme pour le developpement d'applications d'entreprise **distri-**
buees, robustes, securisees et evolutives.

Concept	Role
Conteneur Web	Gere Servlets et JSP (Tomcat)
Conteneur EJB	Gere les composants metier
Services	JNDI (nommage), JTA (transactions), securite, pooling
API principales	Servlets, JSP, JPA, JDBC, JMS

IMPORTANT

A retenir pour l'examen : Un serveur d'application (GlassFish, WildFly, WebLogic) contient TOUS les conteneurs. Tomcat n'est qu'un conteneur WEB (pas complet Java EE).

3 LES SERVIETS (Cours + TPs 2, 3, 4, 5)

3.1 Cycle de vie d'une Servlet - TRES IMPORTANT

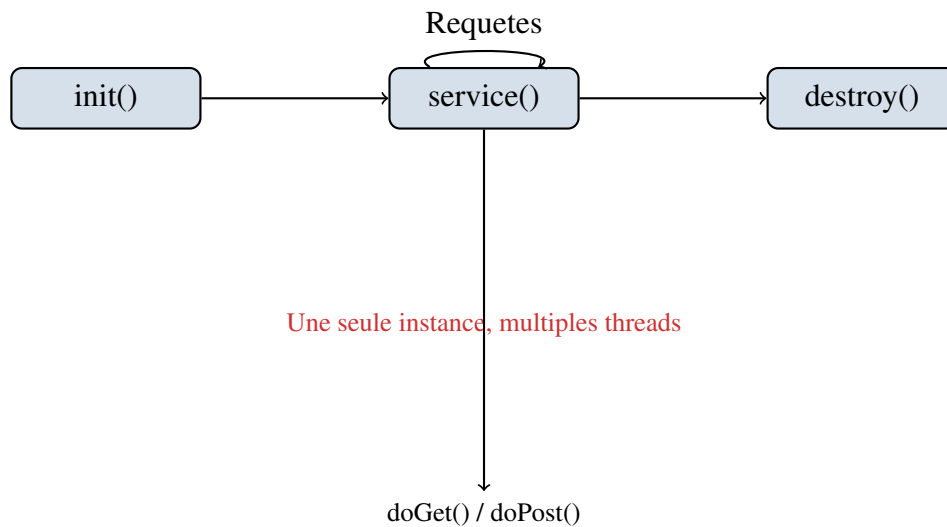


FIGURE 1 – Cycle de vie d'une Servlet

Methode	Quand est-elle appelee ?	Utilite
init()	Une fois au chargement	Initialiser des ressources (connexion BDD)
service()	A chaque requete	Distribue vers doGet/doPost
doGet()	Requete HTTP GET	Recuperer des donnees
doPost()	Requete HTTP POST	Envoyer/modifier des donnees
destroy()	Une fois a l'arret	Liberer les ressources

3.2 Configuration d'une Servlet

Facon 1 - Annotation (recommande) :

```

1 @WebServlet(name = "MaServlet", urlPatterns = {"/accueil", "/home"})
2 public class MaServlet extends HttpServlet { ... }

```

Facon 2 - web.xml :

```

1 <servlet>
2   <servlet-name>MaServlet</servlet-name>
3   <servlet-class>monpackage.MaServlet</servlet-class>
4 </servlet>
5 <servlet-mapping>
6   <servlet-name>MaServlet</servlet-name>
7   <url-pattern>/accueil</url-pattern>
8 </servlet-mapping>

```

3.3 Les methodes HTTP - A COMPARER

Comparaison

Methode	Usage	Parametres	Idempotent	Securite
GET	Recuperer	URL	Oui	Non (visible)
POST	Creer	Corps	Non	Oui
PUT	Remplacer totalement	Corps	Oui	Oui
DELETE	Supprimer	URL	Oui	Oui

A retenir pour l'examen

Difference POST vs PUT : POST = CREER une nouvelle ressource. PUT = REMPLACER complètement une ressource existante.

3.4 Les objets implicites

Objet	Type	Role
request	HttpServletRequest	Parametres, attributs, session
response	HttpServletResponse	Reponse, redirection
session	HttpSession	Stockage entre requetes
out	PrintWriter	Ecrire la reponse HTML

3.5 forward() vs sendRedirect() - A CONNAITRE

Comparaison

forward()	sendRedirect()
Cote SERVEUR	Cote CLIENT (navigateur)
Une seule requete HTTP	Deux requetes HTTP
URL ne change PAS	URL CHANGE
Attributs request CONSERVES	Attributs request PERDUS
Plus RAPIDE	Plus LENT
Limite au contexte	Peut rediriger vers URL EXTERNE

IMPORTANT

A retenir : En MVC, on utilise forward() pour transmettre les donnees du controleur a la vue.

4 JSP (JavaServer Pages) (Cours + TP4)

4.1 Difference Servlet vs JSP

Comparaison

Servlet	JSP
Code Java pur	HTML + Java
Role de CONTROLEUR	Role de VUE
Genere HTML via out.println()	Integre le HTML directement
Convertie en JSP	Convertie en Servlet

4.2 Les Tags JSP

Tag	Syntaxe	Utilisation
Directive <pageDirective>	<pageDeclaration> <pageCommentaire>	<pageScriptlet> <page>

4.3 Les objets implicites JSP

Objet	Equivalent Servlet	Utilite
request	HttpServletRequest	Requete HTTP
response	HttpServletResponse	Reponse HTTP
session	HttpSession	Session utilisateur
application	ServletContext	Contexte global
out	PrintWriter	Flux de sortie
pageContext	-	Contexte de la page

4.4 JSTL

Pour l'importer :

```

1 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2 <!-- Pour Tomcat 10 : -->
3 <%@ taglib uri="jakarta.tags.core" prefix="c" %>

```

Tag	Utilisation
<c:forEach>	Boucle
<c:if>	Condition
<c:choose>, <c:when>, <c:otherwise>	Switch/case
<c:set>	Definir une variable
<c:out>	Afficher (avec echappement)
<c:url>	Construire une URL

A retenir pour l'examen

Role du JSTL : Eviter les scriptlets Java dans les JSP (separation des preoccupations).

5 ARCHITECTURE MVC (Cours + TP4, TP5)

5.1 Definition - QUESTION FREQUENTE

Definition

MVC (Modele-Vue-Contrôleur) separe l'application en 3 parties :

- **Modele (Model)** : Donnees et logique metier
- **Vue (View)** : Presentation (HTML, JSP)
- **Contrôleur (Controller)** : Orchestration des requetes

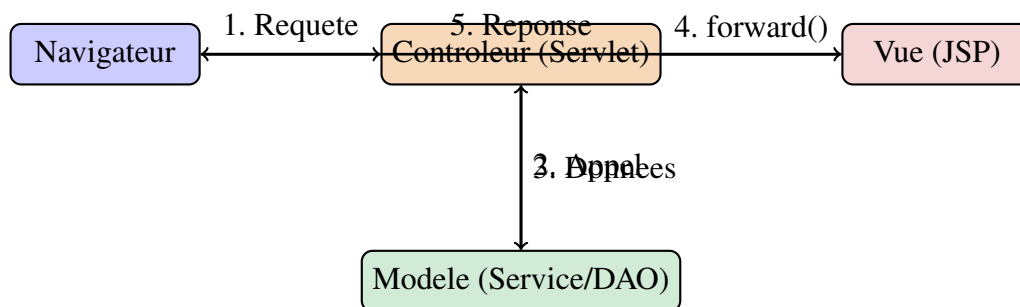


FIGURE 2 – Flux MVC

5.2 Role de chaque couche

Couche	Responsabilite	Annotation Spring
Presentation (Contrôleur)	Recevoir requetes, choisir vue	@Controller / @RestController
Service	Logique metier, transactions	@Service
DAO/Repository	Acces aux donnees	@Repository

5.3 Pourquoi 3 couches ? - QUESTION COURS

A retenir pour l'examen

- **Separation des preoccupations (SoC)**
- **Reutilisabilite** (une DAO pour plusieurs services)
- **Testabilite** (tester chaque couche independamment)
- **Maintenabilite** (changer une couche sans impacter les autres)
- **Flexibilite** (changer de BDD sans toucher a la presentation)

6 JDBC (Cours)

6.1 Les 4 classes principales

Classe	Role
DriverManager	Gere les pilotes, etablit la connexion
Connection	Session avec la BDD
Statement / PreparedStatement	Execute les requetes SQL
ResultSet	Parcourt les resultats

6.2 Statement vs PreparedStatement - TRES IMPORTANT

Comparaison

Statement	PreparedStatement
SQL concatene	SQL avec parametres ?
Vulnerable aux injections SQL	Protege contre les injections
Non precompile	PRECOMPILE (plus performant)
Recompile a chaque execution	Compile une seule fois

Code source

```

1 // NON SECURISE
2 Statement stmt = conn.createStatement();
3 String sql = "SELECT * FROM users WHERE name = '" + userInput + "'";
4
5 // SECURISE
6 PreparedStatement pstmt = conn.prepareStatement(
7     "SELECT * FROM users WHERE name = ?"
8 );
9 pstmt.setString(1, userInput);

```

6.3 Gestion des transactions

```

1 conn.setAutoCommit(false);
2 try {
3     // operations BDD
4     conn.commit();
5 } catch (SQLException e) {
6     conn.rollback();
7 }

```


7 JPA (Cours + TP8)

7.1 JPA vs JDBC vs Hibernate - COMPARAISON

Comparaison

JPA	JDBC	Hibernate
Specification (interface)	API bas niveau	Implementation de JPA
Mapping objet-relationnel	SQL manuel	Framework ORM
Pas de SQL a ecrire	SQL a ecrire	+ fonctionnalites supplementaires

7.2 Annotations JPA fondamentales

Annotation	Role
@Entity	Classe = table BDD
@Table(name = "...")	Nom de la table
@Id	Cle primaire
@GeneratedValue(strategy = ...)	Auto-incrementation
@Column(name = "...")	Mapping colonne
@Transient	Non persiste
@Temporal(TemporalType.DATE)	Date SQL

7.3 Strategies @GeneratedValue - COMPARER

Strategie	Fonctionnement	Portabilite
AUTO	Hibernate choisit	Portable
IDENTITY	Auto-increment SQL (MySQL)	Pas toujours portable
SEQUENCE	Utilise une sequence (Oracle)	Pas toutes les BDD
TABLE	Table speciale pour les IDs	Portable mais lent

7.4 Relations JPA

Annotation	Relation	Cote
@ManyToOne	Plusieurs → Un	L'enfant
@OneToMany(mappedBy = "..")	Un → Plusieurs	Le parent
@OneToOne	Un → Un	-
@ManyToMany	Plusieurs → Plusieurs	Table de jointure

Code source

```

1 @Entity
2 public class Article {
3     @Id @GeneratedValue
4     private Long id;
5
6     @ManyToOne
7     @JoinColumn(name = "categorie_id")
8     private Categorie categorie;
9 }
10
11 @Entity
12 public class Categorie {
13     @Id @GeneratedValue
14     private Long id;
15
16     @OneToMany(mappedBy = "categorie")
17     private List<Article> articles;
18 }

```

7.5 Strategies d'héritage - CODE A SAVOIR

Code source

```

1 @Entity
2 @Inheritance(strategy = InheritanceType.JOINED)
3 public class Personne {
4     @Id @GeneratedValue
5     private Long id;
6     private String nom;
7 }
8
9 @Entity
10 public class Etudiant extends Personne {
11     private String matricule; // Ne pas remettre @Id !
12 }

```

7.6 Role d'EntityManager

Methode	Role
<code>persist()</code>	Insérer
<code>merge()</code>	Mettre à jour
<code>find()</code>	Rechercher par ID
<code>remove()</code>	Supprimer
<code>createQuery()</code>	Requête JPQL

7.7 Le fichier persistence.xml

```

1 <persistence-unit name="monUnite" transaction-type="RESOURCE_LOCAL">
2   <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
3   <properties>
4     <property name="javax.persistence.jdbc.url" value="jdbc:mysql://
localhost:3306/db"/>
5     <property name="hibernate.hbm2ddl.auto" value="update"/>
6     <property name="hibernate.show_sql" value="true"/>
7   </properties>
8 </persistence-unit>

```

8 SPRING BOOT (Cours + TP6, TP7, TP8)

8.1 Definition

Definition

Spring Boot est une extension de Spring qui simplifie la configuration avec :

- **Auto-configuration**
- **Starters** (dependances pre-packaged)
- **Serveur embarque** (Tomcat)
- **Actuator** (monitoring)

8.2 Starters Spring Boot - A CONNAITRE

Starter	Utilite
spring-boot-starter-web	Applications web (MVC, Tomcat)
spring-boot-starter-data-jpa	JPA + Hibernate
spring-boot-starter-security	Securite
spring-boot-starter-test	Tests (JUnit, Mockito)
spring-boot-starter-thymeleaf	Vues Thymeleaf

8.3 L'annotation @SpringBootApplication

A retenir pour l'examen

Elle combine 3 annotations :

1. **@Configuration** - Classe de configuration
2. **@EnableAutoConfiguration** - Auto-configuration Spring Boot
3. **@ComponentScan** - Scanne les composants dans le package

8.4 Les annotations Spring - QCM FREQUENT

Annotation	Role
@Controller	Contrôleur MVC (renvoie vue)
@RestController	Contrôleur REST (renvoie JSON)
@Service	Couche métier
@Repository	Couche DAO
@Autowired	Injection automatique de dépendances
@GetMapping / @PostMapping	Mapper une requête HTTP
@RequestBody	Corps de la requête → objet Java
@PathVariable	Variable d'URL
@Transactional	Gérer les transactions

8.5 Difference @Controller vs @RestController

Comparaison

@Controller	@RestController
Retourne une VUE (JSP/Thymeleaf)	Retourne du JSON/XML
Utilise @ResponseBody pour renvoyer des données	@ResponseBody implicite

8.6 @Autowired - COMPRENDRE LE PRINCIPE

Code source

```

1 // Injection par constructeur (RECOMMANDEE)
2 @Service
3 public class UserService {
4     private final UserRepository repository;
5
6     public UserService(UserRepository repository) {
7         this.repository = repository;
8     }
9 }

```

Spring injecte AUTOMATIQUEMENT le bean de type UserRepository.

8.7 Couche Repository avec Spring Data JPA - TRES IMPORTANT

Code source

```
1 @Repository
2 public interface EmployeeRepository extends JpaRepository<Employee,
   Long> {
3     // Methodes derivees (generees automatiquement)
4     List<Employee> findBySalaryGreaterThan(double salary);
5     List<Employee> findByNameContaining(String name);
6
7     // Requete personnalisee avec @Query
8     @Query("SELECT e FROM Employee e WHERE e.salary > :min")
9     List<Employee> findWithMinSalary(@Param("min") double min);
10 }
```

8.8 Le DTO (Data Transfer Object) - IMPORTANT

A retenir pour l'examen

Pourquoi utiliser un DTO ? Pour NE PAS exposer les champs sensibles (password, createdAt) de l'entite.

Code source

```
1 // Entite (avec champs sensibles)
2 @Entity
3 public class Employee {
4     private String name;
5     private String password; // SENSIBLE - a cacher
6     private Date createdAt;  // INTERNE - a cacher
7 }
8
9 // DTO (ce qui est expose)
10 public class EmployeeDTO {
11     private Long id;
12     private String name;
13     private double salary;
14 }
```

9 CODE DE STATUT HTTP - QCM

Code	Signification
200	OK (succes)
201	Created (cree)
301	Redirection permanente
302	Redirection temporaire
400	Requete invalide
401	Non authentifie
403	Non autorise
404	Ressource non trouvee
405	Methode non supportee par l'URL (TOMBE SOUVENT)
500	Erreur serveur interne

10 SESSIONS HTTP (TP3-2)

10.1 Recuperer / creer une session

```

1 // Recupere la session existante ou en cree une nouvelle
2 HttpSession session = request.getSession();
3
4 // Recupere la session existante (NE PAS en creer de nouvelle)
5 HttpSession session = request.getSession(false);

```

10.2 Utiliser la session

```

1 // Stocker
2 session.setAttribute("user", username);
3 session.setAttribute("compteur", compteur);
4
5 // Lire
6 String user = (String) session.getAttribute("user");
7 Integer compteur = (Integer) session.getAttribute("compteur");
8
9 // Supprimer une session (deconnexion)
10 session.invalidate();

```

A retenir pour l'examen

Pourquoi la session ? HTTP est **stateless** (sans etat). La session permet de garder des informations ENTRE les requetes d'un meme utilisateur.

11 COLLABORATION ENTRE SERVLETS (TP3-2)

11.1 getRequestDispatcher et forward()

```

1 RequestDispatcher rd = request.getRequestDispatcher("/destination.jsp");
2 rd.forward(request, response); // Delegee completement

```

11.2 include()

```

1 RequestDispatcher rd = request.getRequestDispatcher("/fragment.jsp");
2 rd.include(request, response); // Inclut le contenu sans deleguer

```

11.3 Difference forward() vs include()

Comparaison

forward()	include()
Delegee TOTALEMENT	INCLUT du contenu
Le reste du code N'EST PAS execute	Le reste du code EST execute

12 FICHER web.xml (EXAMEN)

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
3     <!-- Page d'accueil -->
4     <welcome-file-list>
5         <welcome-file>index.jsp</welcome-file>
6     </welcome-file-list>
7
8     <!-- Configuration Servlet (alternative aux annotations) -->
9     <servlet>
10         <servlet-name>MaServlet</servlet-name>
11         <servlet-class>monpackage.MaServlet</servlet-class>
12     </servlet>
13     <servlet-mapping>
14         <servlet-name>MaServlet</servlet-name>
15         <url-pattern>/accueil</url-pattern>
16     </servlet-mapping>
17
18     <!-- Page d'erreur -->
19     <error-page>
20         <exception-type>java.lang.Exception</exception-type>
21         <location>/view/erreur.jsp</location>
22     </error-page>
23 </web-app>

```

13 RESUME - CHECKLIST POUR L'EXAMEN

13.1 Questions de cours - PREPAREZ CES REPONSES

A retenir pour l'examen

1. **Definir Java EE** → Plateforme pour applications d'entreprise distribuees
2. **Role du conteneur web** → Gere Servlets/JSP, cycle de vie, mapping URLs
3. **Cycle de vie d'une Servlet** → `init()` → `service()` → `destroy()`
4. **Difference GET/POST/PUT** → GET=lecture, POST=creation, PUT=remplacement
5. **Configuration d'une Servlet** → Annotation `@WebServlet` ou `web.xml`
6. **`forward()` vs `sendRedirect()`** → `forward` serveur, `sendRedirect` client
7. **Difference JPA vs JDBC** → JPA=ORM, JDBC=bas niveau
8. **Difference JPA vs Hibernate** → JPA=specification, Hibernate=implementation
9. **Strategies d'heritage JPA** → JOINED, SINGLE_TABLE, TABLE_PER_CLASS
10. **Relations JPA** → `@ManyToOne`, `@OneToMany` (`mappedBy`)
11. **`@SpringBootApplication`** → `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`
12. **Role de `@Autowired`** → Injection automatique de dependances
13. **Role de `@Query`** → Requetes personnalisées dans Spring Data
14. **Les Spring Boot Starters** → `spring-boot-starter-web`, `data-jpa`, `security`, `test`
15. **Couche Service** → Logique métier, transactions, lien Controller-DAO
16. **Pourquoi 3 couches** → Separation, reutilisabilite, testabilite
17. **Codes HTTP** → 200, 404, 405, 500
18. **Session HTTP** → Stateless, `getSession()`, `setAttribute()`

13.2 SCHEMAS A SAVOIR DESSINER

A retenir pour l'examen

1. Cycle de vie d'une Servlet
2. Architecture MVC (3 couches)
3. Difference `forward()` vs `sendRedirect()`
4. Relations JPA (`ManyToOne`, `OneToMany`)
5. Architecture Spring Boot

13.3 CODES A SAVOIR ECRIRE

A retenir pour l'examen

1. Servlet avec annotation @WebServlet
2. Configuration web.xml
3. Session HTTP (getSession, setAttribute, invalidate)
4. Entite JPA (@Entity, @Id, @GeneratedValue)
5. Relation JPA (@ManyToOne, @OneToMany)
6. Heritage JPA (JOINED)
7. Repository Spring Data JPA
8. Controller REST (@RestController, @GetMapping, @PostMapping)
9. DTO (pour ne pas exposer les champs sensibles)

13.4 Tableau recapitulatif des differences

Comparaison	Point cle
GET vs POST	GET=URL / POST=corps
POST vs PUT	POST=creer / PUT=remplacer totalement
forward() vs sendRedirect()	forward=serveur / sendRedirect=client
Statement vs PreparedStatement	Statement=vulnerable / PreparedStatement=securise
JPA vs JDBC	JPA=ORM haut niveau / JDBC=bas niveau
JPA vs Hibernate	JPA=specification / Hibernate=implementation
Servlet vs JSP	Servlet=controleur / JSP=vue
JOINED vs SINGLE_TABLE	JOINED=tables separees / SINGLE_TABLE=une seule table

BONNE CHANCE POUR L'EXAMEN !